Interview Knowledge Base

RAG Chatbot Edition

HR, Python, SQL, DSA, ML, Deep Learning, NLP, LLMs, RAG, LangChain, System Design, DevOps, CS Core, Statistics, Projects

346 questions and answers across 19 sections

How to use this document

This PDF is a single knowledge base for an interview-preparation chatbot. Every question is a self-contained unit: a clear question followed by a complete answer. That makes it easy to chunk, embed and retrieve in a RAG pipeline.

Suggested ingestion: split on the question headings (each 'Q' starts a new chunk), store the section name as metadata, and keep the question text inside the chunk so semantic search matches user queries well.

For human reading: use the table of contents, read one section a day, and practise answering aloud before reading the model answer. Replace the sample project answers with your own real numbers and experience.

Note: answers are written in simple English for interview use. Verify version-specific details (library APIs, model names, pricing) against current official documentation before relying on them.

Table of Contents

1. HR and Behavioral Questions 4
2. Python Interview Questions 7
3. SQL and Database Questions 11
4. Data Structures and Algorithms 13
5. Machine Learning Fundamentals 15
6. Deep Learning Questions 18
7. NLP and Transformers 20
8. Large Language Models (LLMs) 22
9. RAG (Retrieval-Augmented Generation) - Core Section 24
10. LangChain, LlamaIndex and Tooling 28
11. System Design, APIs and Deployment 30
12. Git, Cloud, Linux and Networking Basics 32
13. Computer Science Core (OS, DBMS, OOP, Networks) 34
14. Statistics and Math for Data Science 35
15. Projects, Resume and Practical Tips 36
16. General and Situational Questions (Non-Coding) 38
17. Aptitude, Logical and Puzzle Questions 41
18. Current Technology, AI Awareness and Ethics 43
19. Communication, English and Interview Etiquette 45
Appendix A: Quick Revision Cheat Sheet 47
Appendix B: Glossary 48

1. HR and Behavioral Questions

Q. Tell me about yourself.

Use a Present - Past - Future structure in about 60-90 seconds.

- Present: your current role or final-year status, your main skills (for example Python, ML, RAG, SQL) and one headline project.
- Past: relevant education, internships or earlier work that built these skills.
- Future: what you want to do next and why this company/role fits.

Example: "I am a final-year B.Tech student who builds AI applications in Python. Recently I built a RAG chatbot that answers questions from PDFs using embeddings, a vector database and an LLM. I enjoy turning messy data into useful products, and I am looking for an AI/ML engineer role where I can build production LLM systems."

Q. Why should we hire you?

Connect three things: (1) skills that match the job description, (2) proof through projects or measurable results, (3) attitude - fast learner, ownership, teamwork. Avoid generic claims; give one concrete example, such as "I reduced chatbot hallucinations by adding retrieval and citation checks."

Q. What are your strengths?

Pick 2-3 strengths that are relevant and back each with a short story. Examples: problem solving, learning quickly, consistency, communication, debugging patiently. Always attach evidence: "I learned LangChain and vector databases in 3 weeks and shipped a working project."

Q. What are your weaknesses?

Choose a real but non-critical weakness and show how you are improving it. Example: "I used to spend too long perfecting code before showing it. Now I share early prototypes and ask for feedback in daily check-ins." Do not say "I have no weakness" or "I work too hard".

Q. Where do you see yourself in 5 years?

Show growth and commitment without sounding fixed on a specific title. "I want to become a strong engineer who owns end-to-end AI systems, mentors juniors and contributes to the product's technical direction."

Q. Why do you want to join our company?

Research the company: product, tech stack, recent news, values. Mention one or two specifics and link them with your interests. Never give salary or location as the first reason.

Q. Why are you leaving your current job?

Stay positive and forward-looking: looking for bigger challenges, learning opportunities, new tech or better alignment with career goals. Never badmouth the old employer or manager.

Q. Describe a challenge you faced and how you handled it.

Use the STAR method.

- Situation: brief context.
- Task: what you were responsible for.
- Action: the specific steps YOU took.
- Result: measurable outcome and lesson learned.

Q. Tell me about a time you failed.

Pick a genuine, modest failure. Explain what went wrong, what you owned, what you changed afterward and a later success that came from the lesson. Interviewers look for accountability, not perfection.

Q. How do you handle pressure and tight deadlines?

Prioritise tasks, break work into small milestones, communicate early if scope or timeline is at risk, and focus on delivering the most valuable part first. Give an example from college or work.

Q. How do you handle conflict with a teammate?

Listen first, focus on the problem not the person, find common goals, propose data-backed options, and involve a lead only if needed. End with the outcome: a better working relationship and a better solution.

Q. Are you comfortable working in a team or alone?

Both. Explain you can own tasks independently but value code reviews, pair debugging and shared documentation. Mention a team project where your role mattered.

Q. What are your salary expectations?

If possible, ask for the budgeted range first. Otherwise give a researched range based on market data, role and location, and say you are flexible for the right opportunity and growth. As a fresher, show you care about learning and role fit as much as pay.

Q. Do you have any questions for us?

Always ask 2-3 good questions.

- What does success look like in the first 90 days?
- What does the team's tech stack and development process look like?
- How does the team evaluate and monitor its AI/ML systems in production?
- What learning opportunities or mentoring exist?

Q. Are you willing to relocate or work in shifts?

Be honest. If flexible, say so clearly. If there are constraints, state them politely and mention alternatives such as remote or hybrid.

Q. What motivates you?

Solving real problems, seeing users benefit from what you build, and continuous learning. Give a small story, for example the first time your chatbot answered a user's question correctly from their own documents.

Q. How do you keep your skills up to date?

Reading papers and blogs, following release notes of tools like LangChain/LlamaIndex/Hugging Face, building side projects, taking courses, contributing on GitHub and discussing in communities.

Q. Describe your ideal work environment.

Collaborative, with clear goals, honest feedback, room to experiment, and respect for work-life balance. Keep it aligned with what you learned about the company.

Q. What is your biggest achievement?

Pick one with measurable impact: a project, hackathon, competition or internship result. Describe the challenge, your contribution and the numbers (accuracy improved, time saved, users served).

Q. How do you prioritise multiple tasks?

Use urgency and impact (Eisenhower matrix), clarify expectations with stakeholders, break tasks down, track in a tool such as Jira/Notion, and review progress daily.

Q. What would you do if you disagree with your manager?

Understand their reasoning, present your viewpoint with data respectfully in private, be open to being wrong, and once a decision is made, commit and support it.

Q. Can you work under minimal supervision?

Yes - explain how you plan work, document assumptions, ask targeted questions early, and give regular updates so no one is surprised.

2. Python Interview Questions

Q. What are Python's key features?

Interpreted, dynamically typed, high-level, object-oriented and multi-paradigm, with a huge standard library and ecosystem (NumPy, Pandas, PyTorch, FastAPI). Memory is managed automatically with reference counting plus a garbage collector.

Q. Difference between list, tuple, set and dictionary?

- List: ordered, mutable, allows duplicates, [1,2,3].
- Tuple: ordered, immutable, allows duplicates, (1,2,3); hashable if its items are hashable.
- Set: unordered, mutable, unique items only, {1,2,3}; average O(1) membership test.
- Dict: key-value mapping with unique keys; insertion ordered since Python 3.7.

Q. What is the difference between mutable and immutable types?

Mutable objects (list, dict, set) can be changed in place; immutable objects (int, float, str, tuple, frozenset, bytes) cannot - any change creates a new object. Immutability makes objects hashable and safe as dictionary keys.

Q. Explain shallow copy vs deep copy.

A shallow copy (copy.copy) creates a new outer object but shares references to nested objects. A deep copy (copy.deepcopy) recursively copies everything, so nested objects are independent. Modifying a nested list in a shallow copy also changes the original.

Q. What are *args and **kwargs?

*args collects extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dict. They make functions flexible: def f(*args,** kwargs).

Q. What is a decorator?

A function that takes another function and returns an enhanced version without modifying its source. Used for logging, timing, caching (functools.lru_cache), authentication and route registration (Flask/FastAPI). Syntax: @decorator above a function.

Q. What are generators and why use them?

Functions using yield that return an iterator and produce values lazily, one at a time. They save memory for large data (for example streaming a huge file or document chunks) and can represent infinite sequences.

Q. Difference between iterator and iterable?

An iterable has __iter__ and can be looped over (list, str). An iterator has __next__ and remembers its position; iter(iterable) returns an iterator.

Q. What is the GIL?

The Global Interpreter Lock in CPython allows only one thread to execute Python bytecode at a time. CPU-bound work does not speed up with threads; use multiprocessing or native extensions. I/O-bound work (API calls, file reads) still benefits from threading or asyncio.

Q. Difference between multithreading, multiprocessing and asyncio?

Threading: shared memory, good for I/O-bound tasks, limited by the GIL for CPU work. Multiprocessing: separate processes and memory, true parallelism for CPU-bound tasks. Asyncio: single-threaded cooperative concurrency using async/await, ideal for many network calls such as concurrent LLM API requests.

Q. What is the difference between == and is?

== compares values (equality); is compares object identity (same memory object). Use "is" for None checks: if x is None.

Q. What are list comprehensions?

A concise way to build lists: [x*x for x in range(10) if x % 2 == 0]. Similar forms exist for sets, dicts and generator expressions. They are usually faster and more readable than equivalent loops.

Q. What is a lambda function?

An anonymous single-expression function: square = lambda x: x*x. Commonly used with map, filter, sorted(key=...).

Q. Explain Python memory management.

Objects live on a private heap managed by the interpreter. Reference counting frees objects when their count hits zero, and a cyclic garbage collector handles reference cycles. Small-object allocation uses pymalloc.

Q. What is __init__ vs __new__?

__new__ creates the instance (a static method returning the object); __init__ initialises it after creation. Override __new__ mainly for immutable types and singletons.

Q. What are dunder (magic) methods?

Special methods with double underscores that customise behaviour: __str__, __repr__, __len__, __getitem__, __eq__, __add__, __enter__/__exit__ (context managers), __call__.

Q. What is a context manager?

An object that sets up and cleans up resources using the with statement: with open("f.txt") as f. Implement via __enter__/__exit__ or the contextlib.contextmanager decorator.

Q. Explain @staticmethod, @classmethod and instance methods.

Instance methods receive self and operate on an object. Class methods receive cls and operate on the class (alternative constructors). Static methods receive neither and are just namespaced functions.

Q. What is inheritance, polymorphism, encapsulation and abstraction?

Inheritance: a class reuses another class's behaviour. Polymorphism: same interface, different implementations. Encapsulation: bundling data with methods and restricting direct access (_protected, __private by convention and name mangling). Abstraction: exposing essential behaviour while hiding details (abc.ABC).

Q. What is multiple inheritance and MRO?

A class can inherit from multiple parents. Method Resolution Order (C3 linearisation) decides the search order; inspect it with ClassName.__mro__. super() follows this order.

Q. Difference between module and package?

A module is a single .py file; a package is a directory of modules containing __init__.py (optional in modern Python with namespace packages).

Q. How do you handle exceptions?

try / except / else / finally. except catches specific errors, else runs if no error occurred, finally always runs. Raise custom exceptions by subclassing Exception. Catch specific exceptions rather than a bare except.

Q. What is a virtual environment?

An isolated Python environment with its own installed packages, preventing dependency conflicts between projects. Create with python -m venv venv, or use conda/poetry/uv.

Q. What is pip freeze and requirements.txt?

pip freeze lists installed packages with versions; requirements.txt stores them so others can reproduce the environment with pip install -r requirements.txt.

Q. Explain Python's type hints.

Optional annotations such as def f(x: int) -> str. They are not enforced at runtime but help IDEs, mypy and libraries like Pydantic/FastAPI validate and document data.

Q. What is a dataclass?

A decorator (@dataclass) that auto-generates __init__, __repr__, __eq__ etc. for classes that mainly store data, reducing boilerplate.

Q. What is the difference between append, extend and insert?

append adds a single element at the end; extend adds all elements of an iterable; insert(i, x) adds at a position (O(n)).

Q. Time complexity of common operations?

List: index O(1), append amortised O(1), insert/delete in middle O(n), search O(n). Dict/set: average O(1) get/set/contains. Sorting: O(n log n) with Timsort.

Q. What is monkey patching?

Modifying a class or module at runtime. Useful in testing (mock objects) but risky in production because it hides behaviour changes.

Q. What are Python's common coding questions?

Reverse a string, check palindrome, Fibonacci, factorial, find duplicates, two-sum, anagram check, flatten nested list, count word frequency, FizzBuzz, merge sorted lists, prime check. Example:

- Reverse string: s[::-1]
- Palindrome: s == s[::-1]
- Word frequency: collections.Counter(text.split())
- Two sum: use a dict of seen values for O(n).

Q. What is the difference between pass, continue and break?

pass does nothing (placeholder); continue skips to the next loop iteration; break exits the loop.

Q. What is pickling?

Serialising Python objects to bytes (pickle.dumps) and restoring them (pickle.loads). Never unpickle data from untrusted sources because it can execute arbitrary code; prefer JSON for interchange.

Q. What is a closure?

A function that remembers variables from its enclosing scope even after that scope finished executing. Basis of decorators and factory functions.

Q. What is the difference between deepcopy and assignment?

Assignment (b = a) makes both names point to the same object; no copy is made.

3. SQL and Database Questions

Q. What is SQL and what are its sub-languages?

Structured Query Language for relational databases. DDL (CREATE, ALTER, DROP, TRUNCATE), DML (INSERT, UPDATE, DELETE), DQL (SELECT), DCL (GRANT, REVOKE), TCL (COMMIT, ROLLBACK, SAVEPOINT).

Q. Difference between DELETE, TRUNCATE and DROP?

DELETE removes selected rows, can use WHERE and can be rolled back. TRUNCATE removes all rows quickly, resets identity and usually cannot be rolled back in many engines. DROP removes the whole table structure and data.

Q. Explain the types of JOINs.

- INNER JOIN: only matching rows in both tables.
- LEFT JOIN: all rows from the left table plus matches from the right (NULL if none).
- RIGHT JOIN: the mirror of LEFT JOIN.
- FULL OUTER JOIN: all rows from both tables.
- CROSS JOIN: Cartesian product.
- SELF JOIN: a table joined with itself.

Q. WHERE vs HAVING?

WHERE filters rows before grouping; HAVING filters groups after GROUP BY and can use aggregate functions such as COUNT(*) > 5.

Q. What is a primary key vs foreign key vs unique key?

Primary key uniquely identifies each row (not null, one per table). Foreign key references a primary key in another table to enforce referential integrity. Unique key enforces uniqueness but allows a NULL (engine-dependent) and a table can have several.

Q. What is normalization?

Organising data to reduce redundancy and anomalies. 1NF: atomic values. 2NF: 1NF plus no partial dependency on part of a composite key. 3NF: 2NF plus no transitive dependency. BCNF is a stricter 3NF. Denormalisation is sometimes used for read performance.

Q. What are ACID properties?

Atomicity (all or nothing), Consistency (valid state to valid state), Isolation (concurrent transactions do not interfere), Durability (committed data survives crashes).

Q. What is an index and when not to use it?

A data structure (often B-tree) that speeds up lookups at the cost of extra storage and slower writes. Avoid on small tables, columns with very low cardinality, or tables with heavy writes and few reads.

Q. Clustered vs non-clustered index?

A clustered index defines the physical order of rows (one per table). A non-clustered index is a separate structure with pointers to rows (many allowed).

Q. What are window functions?

Functions computed across a set of rows related to the current row without collapsing them: ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), SUM() OVER (PARTITION BY ... ORDER BY ...).

Q. How do you find the second highest salary?

SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp); or use DENSE_RANK() OVER (ORDER BY salary DESC) and pick rank 2. For the Nth highest use DENSE_RANK or LIMIT/OFFSET.

Q. How do you find duplicate rows?

SELECT col, COUNT(*) FROM t GROUP BY col HAVING COUNT(*) > 1; To delete duplicates keep one row using ROW_NUMBER() over the duplicate key and delete rows with rn > 1.

Q. What is a CTE?

A Common Table Expression (WITH name AS (...)) is a named temporary result set that improves readability and supports recursion.

Q. Difference between UNION and UNION ALL?

UNION removes duplicates (slower, sorts); UNION ALL keeps all rows (faster).

Q. What is a view?

A saved query that behaves like a virtual table. Used for abstraction and security. A materialized view stores the result physically.

Q. What is a stored procedure vs function?

A stored procedure is a saved set of SQL statements that can perform actions and return multiple results; a function returns a value and can be used inside queries.

Q. SQL vs NoSQL?

SQL: structured schema, joins, ACID, vertical scaling traditionally (PostgreSQL, MySQL). NoSQL: flexible schema, horizontal scaling, types include document (MongoDB), key-value (Redis), column (Cassandra), graph (Neo4j). Choose based on data shape, consistency and scale needs.

Q. What is the CAP theorem?

A distributed system can guarantee only two of Consistency, Availability and Partition tolerance at the same time. Since partitions happen, real systems trade off between C and A.

Q. What is a transaction isolation level?

Read Uncommitted, Read Committed, Repeatable Read and Serializable control which anomalies (dirty read, non-repeatable read, phantom read) are allowed. Higher isolation = more safety, less concurrency.

Q. How do you optimise a slow SQL query?

Check the execution plan (EXPLAIN), add proper indexes, avoid SELECT *, filter early, avoid functions on indexed columns in WHERE, reduce joins or subqueries, paginate, and cache frequent results.

Q. What is a vector database and how does it differ from a relational DB?

A vector DB stores high-dimensional embeddings and supports similarity search (nearest neighbours) using ANN indexes such as HNSW or IVF. Relational DBs query exact values with SQL. pgvector adds vector search to PostgreSQL.

4. Data Structures and Algorithms

Q. What is the difference between an array and a linked list?

Arrays: contiguous memory, O(1) index access, O(n) insertion in the middle. Linked lists: nodes with pointers, O(1) insertion given the node, O(n) access.

Q. Stack vs queue?

Stack is LIFO (push/pop), used in recursion, undo, expression parsing. Queue is FIFO (enqueue/dequeue), used in BFS, scheduling, buffering. Deque supports both ends.

Q. What is a hash table and how are collisions handled?

It maps keys to buckets through a hash function for average O(1) operations. Collisions are handled with chaining (linked lists in buckets) or open addressing (linear/quadratic probing, double hashing).

Q. Explain binary search.

Repeatedly halve a sorted search space by comparing the target with the middle element. Time O(log n), space O(1) iterative. Also used on answer spaces ("binary search on answer").

Q. Compare common sorting algorithms.

- Bubble/Insertion/Selection: O(n^2), simple.
- Merge sort: O(n log n) always, stable, O(n) extra space.
- Quick sort: O(n log n) average, O(n^2) worst, in-place.
- Heap sort: O(n log n), in-place, not stable.
- Counting/Radix: linear for bounded integer keys.

Q. BFS vs DFS?

BFS explores level by level using a queue - finds the shortest path in unweighted graphs. DFS goes deep using a stack/recursion - good for cycle detection, topological sort, connected components. Both are O(V+E).

Q. What is a binary search tree? What is a balanced tree?

A BST keeps left < node < right, so search/insert/delete are O(h). Balanced trees (AVL, Red-Black) keep height O(log n) to guarantee O(log n) operations.

Q. What is a heap / priority queue?

A complete binary tree where each parent is >= (max-heap) or <= (min-heap) its children. Insert and extract are O(log n); peek is O(1). Used for top-k, scheduling, Dijkstra.

Q. What is a trie?

A prefix tree storing strings character by character; supports prefix search in O(length). Used in autocomplete and spell checkers.

Q. What is dynamic programming?

Solving problems with overlapping subproblems and optimal substructure by storing results (memoization top-down or tabulation bottom-up). Examples: Fibonacci, knapsack, LCS, coin change, edit distance.

Q. What is the two-pointer / sliding window technique?

Two-pointer uses two indices moving through a sorted array or string to cut O(n^2) to O(n). Sliding window maintains a moving range for subarray/substring problems (longest substring without repeats, max sum of k elements).

Q. How do you detect a cycle in a linked list?

Floyd's tortoise-and-hare: a slow pointer moves 1 step, a fast pointer moves 2; if they meet, a cycle exists. O(n) time, O(1) space.

Q. Explain Dijkstra's algorithm.

Finds the shortest paths from a source in a graph with non-negative weights using a min-heap. Complexity O((V+E) log V). Bellman-Ford handles negative weights in O(VE).

Q. What is Big-O notation?

It describes the upper bound on how an algorithm's time or space grows with input size n. Common classes: O(1), O(log n), O(n), O(n log n), O(n^2), O(2^n), O(n!).

Q. What is recursion and its drawbacks?

A function calling itself with a smaller input until a base case. Drawbacks: stack overflow risk and repeated work (fixed with memoization). Python's default recursion limit is about 1000.

Q. Common DSA problems to practise?

Two Sum, Valid Parentheses, Merge Intervals, Longest Substring Without Repeating Characters, Reverse Linked List, Binary Tree Level Order Traversal, Number of Islands, Top K Frequent Elements, LRU Cache, Climbing Stairs, Coin Change, Kth Largest Element, Course Schedule (topological sort).

Q. How to implement an LRU cache?

Use a hash map plus a doubly linked list (or Python's OrderedDict). get and put are O(1): move the accessed node to the front and evict from the tail when capacity is exceeded.

Q. What is the difference between a process and a thread?

A process has its own memory space; threads share memory within a process. Context switching between threads is cheaper; processes are more isolated.

5. Machine Learning Fundamentals

Q. What is machine learning and its types?

Learning patterns from data to make predictions without explicit rules. Types: supervised (labelled data), unsupervised (no labels - clustering, dimensionality reduction), semi-supervised, self-supervised and reinforcement learning (agent learns through rewards).

Q. Explain the bias-variance trade-off.

Bias is error from overly simple assumptions (underfitting); variance is error from sensitivity to training data (overfitting). Increasing model complexity lowers bias but raises variance. The goal is the sweet spot with the lowest generalisation error.

Q. What is overfitting and how do you prevent it?

The model memorises training data and fails on new data. Prevent via more data, simpler models, regularisation (L1/L2, dropout), cross-validation, early stopping, data augmentation and feature selection.

Q. L1 vs L2 regularisation?

L1 (Lasso) adds the sum of absolute weights and can drive weights to exactly zero (feature selection). L2 (Ridge) adds the sum of squared weights and shrinks weights smoothly. Elastic Net combines both.

Q. Explain precision, recall, F1 and accuracy.

Precision = TP/(TP+FP): of predicted positives, how many are right. Recall = TP/(TP+FN): of actual positives, how many are found. F1 = harmonic mean of precision and recall. Accuracy = correct predictions / all predictions - misleading for imbalanced data.

Q. What is a confusion matrix and ROC-AUC?

A confusion matrix tabulates TP, FP, TN, FN. The ROC curve plots true positive rate against false positive rate across thresholds; AUC is the area under it (1.0 perfect, 0.5 random).

Q. How do you handle imbalanced data?

Resampling (SMOTE oversampling, undersampling), class weights, appropriate metrics (precision-recall, F1), threshold tuning, anomaly-detection framing and collecting more minority data.

Q. What is cross-validation?

Splitting data into k folds, training on k-1 and validating on the remaining fold, rotating through all folds and averaging the score. It gives a more reliable estimate of generalisation.

Q. Explain linear regression and its assumptions.

Models a target as a linear combination of features, minimising squared error. Assumptions: linearity, independence of errors, homoscedasticity, normally distributed residuals and no heavy multicollinearity.

Q. Logistic regression - how does it work?

It applies the sigmoid function to a linear combination of features to output a probability between 0 and 1 and is trained with log loss (cross-entropy). It is a classifier despite the name.

Q. How does a decision tree work?

It recursively splits data on the feature/threshold that gives the best impurity reduction (Gini or entropy / information gain). Easy to interpret but prone to overfitting.

Q. Random forest vs gradient boosting?

Random forest builds many decorrelated trees in parallel on bootstrapped samples and averages them (bagging) - reduces variance. Gradient boosting builds trees sequentially, each correcting the previous errors (XGBoost, LightGBM, CatBoost) - reduces bias and often wins on tabular data.

Q. What is SVM?

Support Vector Machine finds the hyperplane maximising the margin between classes; kernels (RBF, polynomial) handle non-linear boundaries.

Q. What is KNN?

K-Nearest Neighbours classifies a point by majority vote of its k closest training points. Simple, but slow at inference and sensitive to feature scaling.

Q. Explain K-means clustering.

Choose k, initialise centroids, assign points to the nearest centroid, recompute centroids, repeat until convergence. Choose k with the elbow method or silhouette score. Sensitive to initialisation and outliers.

Q. What is PCA?

Principal Component Analysis projects data onto orthogonal directions of maximum variance to reduce dimensionality while retaining most information. Features should be scaled first.

Q. What is feature scaling and why is it needed?

Standardisation (z-score) or normalisation (min-max) puts features on comparable scales. Needed for distance-based and gradient-based models (KNN, SVM, neural nets, regression with regularisation); tree models do not need it.

Q. What is data leakage?

Information from outside the training set (such as test data or future data) leaking into model training, producing overly optimistic results. Prevent by splitting before preprocessing and using pipelines.

Q. Gradient descent - explain.

An optimisation algorithm that updates parameters in the opposite direction of the gradient of the loss, scaled by a learning rate. Variants: batch, stochastic (SGD) and mini-batch; optimisers: Momentum, RMSProp, Adam.

Q. What are hyperparameters and how do you tune them?

Settings chosen before training (learning rate, tree depth, k). Tune with grid search, random search, Bayesian optimisation (Optuna) and validate on a separate set.

Q. How do you handle missing values?

Drop rows/columns if few, impute with mean/median/mode, use model-based imputation (KNN, iterative), add a "missing" indicator, or use algorithms that handle NaN natively (XGBoost, LightGBM).

Q. What is the curse of dimensionality?

As the number of features grows, data becomes sparse and distances lose meaning, so models need exponentially more data. Mitigate with feature selection and dimensionality reduction.

Q. What is the difference between parametric and non-parametric models?

Parametric models have a fixed number of parameters (linear regression); non-parametric models grow in complexity with data (KNN, decision trees).

Q. Generative vs discriminative models?

Discriminative models learn P(y|x) - the decision boundary (logistic regression, SVM). Generative models learn P(x, y) or P(x) and can generate data (Naive Bayes, GANs, diffusion models, LLMs).

6. Deep Learning Questions

Q. What is a neural network?

Layers of connected neurons where each computes a weighted sum plus bias, passed through a non-linear activation. Trained by backpropagation to minimise a loss function.

Q. Explain backpropagation.

Uses the chain rule to compute gradients of the loss with respect to every weight by propagating errors backward from output to input layers; an optimiser then updates the weights.

Q. Common activation functions?

Sigmoid (0-1, vanishing gradients), Tanh (-1 to 1), ReLU (max(0,x), fast, can "die"), Leaky ReLU, GELU (used in Transformers), Softmax (probabilities for multi-class output).

Q. What is the vanishing gradient problem?

In deep networks gradients shrink as they move backward, so early layers learn very slowly. Fixes: ReLU-type activations, residual connections, batch/layer normalisation, careful initialisation, LSTM/GRU gates.

Q. What are CNNs used for?

Convolutional Neural Networks exploit local spatial structure using convolution filters, pooling and weight sharing. Used for images, video and some text/audio tasks. Famous architectures: LeNet, AlexNet, VGG, ResNet, EfficientNet.

Q. RNN vs LSTM vs GRU?

RNNs process sequences with a hidden state but struggle with long dependencies. LSTM adds input, forget and output gates plus a cell state to retain long-term information. GRU is a lighter gated variant. Transformers have largely replaced them in NLP.

Q. What is dropout?

Randomly turning off a fraction of neurons during training so the network does not rely on specific units, reducing overfitting. It is disabled at inference.

Q. What is batch normalisation?

Normalises layer inputs across the mini-batch to stabilise and speed up training, allowing higher learning rates. Layer normalisation (used in Transformers) normalises across features instead.

Q. What are epochs, batches and iterations?

An epoch is one full pass over the training data; a batch is a subset of samples used for one update; iterations per epoch = dataset size / batch size.

Q. What is transfer learning and fine-tuning?

Reusing a model pretrained on a large dataset and adapting it to a new task, either by training only a new head or by fine-tuning some/all layers with a small learning rate. It saves data and compute.

Q. What is the Adam optimiser?

It combines momentum and adaptive per-parameter learning rates (RMSProp-like) using running averages of gradients and squared gradients. AdamW decouples weight decay and is standard for Transformers.

Q. What is a loss function? Examples?

A measure of prediction error to minimise. Regression: MSE, MAE, Huber. Classification: binary and categorical cross-entropy. Others: contrastive and triplet loss for embeddings.

Q. PyTorch vs TensorFlow?

PyTorch uses dynamic graphs, feels Pythonic and dominates research and LLM tooling. TensorFlow/Keras has strong deployment tooling (TF Serving, TFLite). Both are production capable.

Q. What are GANs, autoencoders and diffusion models?

GANs pit a generator against a discriminator. Autoencoders compress and reconstruct data through a bottleneck (VAEs add a probabilistic latent space). Diffusion models learn to denoise data gradually and power modern image generation.

Q. What is the attention mechanism?

It lets a model weigh the importance of different input positions when producing each output. Scaled dot-product attention computes softmax(QK^T / sqrt(d_k)) V, where Q, K and V are query, key and value projections.

7. NLP and Transformers

Q. What is NLP?

Natural Language Processing is the field of making computers understand, interpret and generate human language. Tasks: classification, named entity recognition, translation, summarisation, question answering, search and dialogue.

Q. What is tokenization?

Splitting text into units (tokens) - words, sub-words or characters. Modern LLMs use sub-word schemes such as BPE, WordPiece and SentencePiece to handle rare words and keep vocabulary small. Token count (not word count) determines cost and context usage.

Q. Stemming vs lemmatization?

Stemming chops suffixes by rules (running -> run, but studies -> studi). Lemmatization uses vocabulary and grammar to return the dictionary form (better -> good). Lemmatization is more accurate but slower.

Q. What are stop words?

Very common words (the, is, and) often removed in classical NLP to reduce noise. Not removed for modern Transformer models because they carry context.

Q. Explain TF-IDF.

Term Frequency times Inverse Document Frequency: a word is important if frequent in a document but rare across the corpus. Used for keyword search and classical text classification. BM25 is a stronger variant widely used in search.

Q. What are word embeddings?

Dense vectors representing words so that similar meanings lie close together. Word2Vec, GloVe and FastText give static embeddings; BERT-style models give contextual embeddings where the same word gets different vectors in different sentences.

Q. How does the Transformer architecture work?

It replaces recurrence with self-attention. The encoder stack builds contextual representations; the decoder generates output while attending to previous tokens (masked self-attention) and encoder outputs. Key parts: multi-head attention, feed-forward layers, residual connections, layer norm and positional encodings. Introduced in "Attention Is All You Need" (2017).

Q. What is self-attention and why multi-head?

Self-attention lets every token look at every other token to compute context-aware representations. Multiple heads let the model attend to different relationships (syntax, coreference, semantics) in parallel subspaces.

Q. Why positional encoding?

Attention itself is order-agnostic, so position information (sinusoidal, learned, rotary RoPE or ALiBi) is added so the model knows token order.

Q. BERT vs GPT?

BERT is encoder-only, bidirectional, trained with masked language modelling - great for understanding tasks (classification, embeddings). GPT is decoder-only, autoregressive, predicts the next token - great for generation. T5/BART are encoder-decoder.

Q. What is the context window?

The maximum number of tokens (prompt plus output) a model can process at once. Long contexts cost more and can still suffer "lost in the middle" effects, which is one reason retrieval helps.

Q. What is perplexity?

A measure of how well a language model predicts text; the exponential of average negative log-likelihood. Lower is better.

Q. What is Named Entity Recognition?

Identifying and classifying entities such as person, organisation, location and date in text. Useful for metadata extraction in RAG pipelines.

Q. How do you evaluate NLP models?

Classification: accuracy, F1. Translation: BLEU, COMET. Summarisation: ROUGE, BERTScore. Retrieval: Recall@k, MRR, nDCG. Generation: human evaluation and LLM-as-judge.

8. Large Language Models (LLMs)

Q. What is an LLM?

A Transformer-based neural network with billions of parameters, pretrained on massive text corpora to predict the next token, which gives it broad language, reasoning and coding abilities. Examples: GPT, Claude, Gemini, Llama, Mistral.

Q. How are LLMs trained?

Stage 1: pretraining on web-scale text (next-token prediction). Stage 2: supervised fine-tuning (SFT) on instruction-response pairs. Stage 3: alignment using RLHF, DPO or similar preference methods so outputs are helpful, honest and safe.

Q. What are hallucinations and how do you reduce them?

Hallucinations are fluent but false or unsupported outputs. Reduce them with retrieval-augmented generation, grounding answers in provided context, citations, instructing the model to say "I don't know", lower temperature, output validation, and evaluation with test sets.

Q. What are temperature, top-p and top-k?

Temperature scales randomness (low = deterministic, high = creative). Top-k samples only from the k most likely tokens; top-p (nucleus) samples from the smallest set whose cumulative probability exceeds p. Use low temperature for factual RAG answers.

Q. What is prompt engineering?

Designing inputs to get better outputs: clear instructions, role and context, examples (few-shot), output format constraints, step-by-step reasoning, delimiters and guardrails. Iterate with tests rather than guesswork.

Q. Zero-shot vs few-shot vs chain-of-thought?

Zero-shot: only an instruction. Few-shot: includes examples. Chain-of-thought: encourages step-by-step reasoning before the answer, improving multi-step problems.

Q. What is fine-tuning and when should you use it vs RAG?

Fine-tuning changes model weights to teach style, format or domain behaviour. RAG supplies fresh external knowledge at inference time. Use RAG for changing or private facts and citations; fine-tune for consistent tone, structure or specialised tasks; many systems combine both.

Q. What are LoRA and QLoRA?

LoRA (Low-Rank Adaptation) trains small low-rank adapter matrices instead of all weights, cutting memory and cost. QLoRA does this on a quantised (4-bit) base model so large models can be fine-tuned on a single GPU.

Q. What is quantization?

Reducing numerical precision of weights (FP16 -> INT8 / INT4) to shrink memory and speed up inference with small accuracy loss. Formats and tools: GGUF, GPTQ, AWQ, bitsandbytes.

Q. What is function calling / tool use?

The model returns structured arguments for developer-defined functions (search, calculator, database query), the application executes them and returns results to the model. It enables agents and reliable integrations.

Q. What are AI agents?

LLM-driven systems that plan, choose tools, observe results and iterate toward a goal (ReAct pattern: reason + act). Frameworks include LangGraph, CrewAI and AutoGen. Needs guardrails, limits on steps and monitoring.

Q. What is prompt injection?

An attack where malicious text in user input or retrieved documents tries to override the system instructions. Mitigate with input/output filtering, separating instructions from data, least-privilege tools, human approval for sensitive actions and treating retrieved content as untrusted.

Q. What is RLHF?

Reinforcement Learning from Human Feedback: humans rank model outputs, a reward model learns the preferences, and the LLM is optimised (e.g. with PPO) to maximise that reward. DPO achieves similar alignment without an explicit reward model.

Q. What is a system prompt vs user prompt?

The system prompt sets the model's role, rules and style for the whole session; user prompts contain the task or question. Never put secrets in the system prompt - assume it can leak.

Q. How do you control cost and latency in LLM apps?

Use smaller models for easy tasks, cache responses/embeddings, trim context, stream outputs, batch requests, use prompt caching where supported, and monitor token usage.

Q. What is MoE (Mixture of Experts)?

An architecture with many expert sub-networks where a router activates only a few per token, giving large capacity with lower compute per token.

Q. What is KV cache?

Cached key/value tensors from previous tokens so the model does not recompute attention history at each decoding step; it speeds up generation but uses significant GPU memory.

9. RAG (Retrieval-Augmented Generation) - Core Section

Q. What is RAG?

Retrieval-Augmented Generation combines a retriever (search over your own documents) with a generator (an LLM). For each question the system fetches relevant chunks and puts them into the prompt so the model answers using that evidence, giving up-to-date, source-grounded and less hallucinated responses without retraining the model.

Q. Explain the RAG pipeline end to end.

- Ingestion: load documents (PDF, DOCX, web, DB) and clean text.
- Chunking: split into passages with some overlap.
- Embedding: convert each chunk to a vector using an embedding model.
- Indexing: store vectors plus metadata in a vector database.
- Retrieval: embed the user query and fetch top-k similar chunks (optionally hybrid + reranking).
- Augmentation: build a prompt with instructions, retrieved context and the question.
- Generation: the LLM writes the answer, ideally with citations.
- Evaluation and monitoring: measure retrieval and answer quality continuously.

Q. Why use RAG instead of just a bigger prompt or fine-tuning?

Context windows are limited and costly; stuffing every document is slow and noisy. Fine-tuning is poor at adding precise, changing facts and gives no citations. RAG is cheaper to update (just re-index), supports access control per document and provides traceable sources.

Q. What is chunking and why does it matter?

Splitting documents into smaller pieces that fit the embedding model and capture one idea. Chunks too large dilute meaning and waste context; too small lose context. Typical sizes are 200-800 tokens with 10-20 percent overlap, tuned per use case.

Q. What chunking strategies exist?

Fixed-size, recursive character splitting (respects paragraphs/sentences), sentence-based, semantic chunking (split where topic changes), structure-aware (headings, tables, code blocks), parent-child / small-to-big retrieval (retrieve small chunk, send its larger parent), and sliding window.

Q. Why use chunk overlap?

Overlap prevents important information that sits at a chunk boundary from being cut in half, preserving context across neighbouring chunks.

Q. What is an embedding model?

A model that maps text to a fixed-length dense vector so semantically similar texts are close in vector space. Examples: OpenAI text-embedding-3, Cohere embed, BGE, E5, GTE, Sentence-Transformers (all-MiniLM-L6-v2). Use the same model for documents and queries.

Q. How do you choose an embedding model?

Consider language support (multilingual for Hindi/Hinglish), retrieval benchmark scores (MTEB), vector dimension (storage and speed), max input length, cost/latency, licence, and whether you can host it. Always test on your own data.

Q. What similarity metrics are used?

Cosine similarity (angle between vectors, most common), dot product (equivalent to cosine for normalised vectors) and Euclidean (L2) distance. Pick the metric the embedding model was trained for.

Q. What is a vector database? Name some.

A database optimised to store embeddings and run fast approximate nearest neighbour search with metadata filtering. Examples: FAISS (library), Chroma, Pinecone, Weaviate, Qdrant, Milvus, pgvector, Elasticsearch/OpenSearch, Redis.

Q. What are ANN indexes - HNSW, IVF, PQ?

Exact search over millions of vectors is slow, so ANN trades small accuracy loss for big speed. HNSW is a layered proximity graph (fast, high recall, more memory). IVF clusters vectors and searches only nearby clusters. PQ (product quantisation) compresses vectors to save memory.

Q. What is top-k retrieval and how do you choose k?

Returning the k most similar chunks. Small k may miss evidence; large k adds noise and cost. Start with 3-8, evaluate recall and answer quality, and combine with a reranker so you can retrieve more (say 20-50) and keep the best few.

Q. What is hybrid search?

Combining keyword search (BM25 - exact terms, IDs, names) with vector search (semantic meaning) and merging results, often by Reciprocal Rank Fusion. It improves recall on queries where either method alone fails.

Q. What is reranking?

A second-stage model (cross-encoder such as bge-reranker or Cohere Rerank) scores each query-chunk pair jointly for much better relevance than vector similarity alone. Retrieve broadly first, then rerank to the best few.

Q. What is query transformation?

Improving the query before retrieval: query rewriting, multi-query expansion, step-back prompting, HyDE (generate a hypothetical answer and embed it), decomposition of complex questions into sub-questions, and using chat history to make follow-up questions standalone.

Q. How do you handle multi-turn conversations in a RAG chatbot?

Keep a chat history, rewrite the latest user message into a standalone question using the history (condense step), retrieve with that, and include a short window or summary of history in the generation prompt. Limit memory size to control tokens.

Q. How do you reduce hallucinations in RAG?

Strict prompt ("answer only from the context, otherwise say you do not know"), high-quality retrieval, reranking, citations to chunk sources, relevance thresholds (refuse when similarity is low), low temperature, answer verification / self-check and evaluation.

Q. What are the common failure modes of RAG?

- Bad parsing (tables, scanned PDFs, headers/footers).
- Poor chunking that splits key info.
- Wrong embedding model for the domain/language.
- Retrieval misses (low recall) or noisy chunks (low precision).
- Lost in the middle - the model ignores context in the middle of long prompts.
- Outdated or conflicting documents.

- The model ignoring the context and using prior knowledge.

Q. How do you evaluate a RAG system?

Evaluate retrieval and generation separately. Retrieval: Recall@k, Precision@k, MRR, nDCG, hit rate. Generation: faithfulness/groundedness, answer relevance, context relevance, correctness. Tools: RAGAS, TruLens, DeepEval, LangSmith, Arize Phoenix. Build a golden dataset of real questions with expected answers and sources.

Q. What is RAGAS and what metrics does it have?

An open-source framework for evaluating RAG pipelines. Common metrics: faithfulness (is the answer supported by the context), answer relevancy, context precision, context recall and answer correctness; often uses an LLM as the judge.

Q. How do you handle PDFs with tables, images and scanned pages?

Use layout-aware parsers (PyMuPDF, pdfplumber, Unstructured, LlamaParse, Docling, Azure Document Intelligence), OCR for scanned pages (Tesseract, PaddleOCR), convert tables to markdown/CSV text, caption images with a vision model, and store page numbers and section titles as metadata.

Q. What metadata should you store with chunks?

Source file name, page number, section/heading, document date/version, author, access-control labels, language and chunk id. Metadata enables filtering, citations, freshness control and security.

Q. How do you keep the index up to date?

Use incremental ingestion with content hashes to detect changes, re-embed only modified documents, delete stale vectors by document id, schedule jobs, and version your embedding model (re-index everything when it changes).

Q. Naive RAG vs Advanced RAG vs Modular RAG?

Naive: chunk, embed, retrieve top-k, generate. Advanced: adds query rewriting, hybrid search, reranking, context compression and better chunking. Modular/agentic: routers, multiple retrievers, tool use and iterative retrieval where an agent decides what to search next.

Q. What is Agentic RAG?

An agent decides whether retrieval is needed, which source/tool to query, how to rephrase queries, and whether results are sufficient, looping until it can answer. More flexible but costlier and harder to debug.

Q. What is Graph RAG?

Builds a knowledge graph of entities and relationships from documents and retrieves connected facts or community summaries, helping with multi-hop and "global" questions that plain vector search handles poorly.

Q. What is context compression?

Reducing retrieved text before sending it to the LLM - extracting only relevant sentences, summarising or filtering chunks - to cut tokens and noise.

Q. How do you secure a RAG chatbot?

Enforce document-level access control at retrieval time, filter by user permissions, redact PII, defend against prompt injection in documents, log and audit queries, rate-limit, protect API keys, and avoid sending sensitive data to third-party APIs without agreements.

Q. How do you make RAG responses faster?

Cache embeddings and frequent answers, use an efficient ANN index, limit k and chunk size, run retrieval and reranking in parallel where possible, stream tokens to the user, and use a smaller/faster model for rewriting.

Q. How do you cite sources in RAG?

Keep chunk metadata, number the chunks in the prompt, ask the model to cite [1], [2] for each claim, and map the numbers back to file name and page in the UI. Optionally verify citations programmatically.

Q. What is the "lost in the middle" problem?

LLMs tend to use information at the start and end of a long context better than the middle. Mitigate by reranking, putting the best chunks first/last, and sending fewer, higher-quality chunks.

Q. What happens when no relevant document exists?

Use a similarity score threshold or reranker score to detect low confidence and respond with a graceful fallback such as "I could not find this in the knowledge base", optionally suggesting related topics or escalating to a human.

Q. What is the difference between semantic search and keyword search?

Keyword search matches exact terms (fast, precise for codes and names, misses synonyms). Semantic search matches meaning via embeddings (handles paraphrases, may miss exact identifiers). Hybrid combines both.

Q. What is multi-vector or ColBERT retrieval?

Instead of one vector per chunk, late-interaction models keep a vector per token and score with fine-grained matching, which improves accuracy at higher storage cost.

Q. Describe a RAG project you built in an interview.

Template: "I built a chatbot over [document type]. I parsed PDFs with [tool], chunked with recursive splitting (500 tokens, 50 overlap), embedded with [model] and stored vectors in [Chroma/FAISS/Qdrant]. Queries use hybrid retrieval and reranking, then [LLM] answers with citations. I evaluated with RAGAS on [N] test questions: faithfulness [x], context recall [y]. The main challenges were [tables/latency/hallucination] and I solved them by [action]." Always include numbers and trade-offs.

Q. What improvements would you make to your RAG chatbot next?

Add hybrid search and reranking, evaluation dashboards, user feedback loop (thumbs up/down), semantic caching, query routing across multiple knowledge bases, better table/image handling, access control and monitoring for latency, cost and failed retrievals.

10. LangChain, LlamaIndex and Tooling

Q. What is LangChain?

An open-source framework providing building blocks for LLM applications: model wrappers, prompt templates, document loaders, text splitters, vector store integrations, retrievers, chains, memory, tools and agents. LangGraph (from the same ecosystem) models stateful, multi-step agent workflows as graphs.

Q. What is LlamaIndex?

A data framework focused on connecting LLMs with your data: advanced ingestion, indexing, query engines, routers and evaluation. It is especially strong for RAG; LangChain is broader for general orchestration. Many teams use both or neither.

Q. What is LCEL?

LangChain Expression Language - a declarative way to compose runnables using the pipe operator, e.g. prompt | llm | parser, with built-in streaming, batching and async support.

Q. What is a retriever vs a vector store?

A vector store stores embeddings and performs similarity search. A retriever is a higher-level interface returning relevant documents for a query; it may wrap a vector store, add filters, hybrid logic, multi-query or reranking.

Q. What are text splitters in LangChain?

Utilities that chunk documents; RecursiveCharacterTextSplitter is the common default - it tries to split on paragraphs, then sentences, then words to keep chunks coherent.

Q. What is conversational memory?

Mechanisms to preserve chat context: buffer (full history), window (last k turns), summary (running summary) or vector-backed memory. In modern setups you often store history yourself and pass it to the chain.

Q. What is Hugging Face?

A platform and library ecosystem (Transformers, Datasets, PEFT, Hub) hosting open models and datasets and offering APIs to load, fine-tune and run them.

Q. What is Ollama / vLLM / llama.cpp?

Tools to run LLMs locally or in your own infrastructure. Ollama is easy local serving, llama.cpp runs quantised models efficiently on CPU/GPU, vLLM is a high-throughput GPU inference server using PagedAttention and continuous batching.

Q. How do you build a simple RAG chatbot in Python?

Steps: (1) load PDFs with a loader, (2) split into chunks, (3) create embeddings, (4) store in Chroma/FAISS, (5) create a retriever with k=4, (6) write a prompt with context and question, (7) call the LLM, (8) wrap in a UI (Streamlit/Gradio) or API (FastAPI), (9) add citations and chat history, (10) evaluate and deploy.

Q. What is the difference between a chain and an agent?

A chain follows a fixed, predetermined sequence of steps. An agent lets the LLM decide dynamically which tools to call and in what order based on intermediate results.

Q. What is MCP (Model Context Protocol)?

An open protocol standardising how AI applications connect to external tools and data sources through MCP servers, so one integration works across multiple clients and models.

11. System Design, APIs and Deployment

Q. How would you design a production RAG chatbot?

- Ingestion service: connectors, parsing, chunking, embedding workers and a queue (Celery/Kafka).
- Storage: vector DB, object storage for raw files, relational DB for metadata and users.
- Query service (FastAPI): auth, query rewriting, hybrid retrieval, reranker, prompt builder, LLM call with streaming.
- Cache layer (Redis) for embeddings and frequent answers.
- Observability: logs, tracing (LangSmith/OpenTelemetry), latency/cost dashboards and feedback capture.
- Evaluation pipeline run on every change; CI/CD with Docker and Kubernetes or a managed platform.

Q. What is a REST API?

An architectural style using HTTP methods on resources: GET (read), POST (create), PUT/PATCH (update), DELETE (remove). It is stateless and returns commonly JSON with meaningful status codes (200, 201, 400, 401, 403, 404, 429, 500).

Q. FastAPI vs Flask vs Django?

FastAPI: modern, async, automatic validation via Pydantic and OpenAPI docs - great for ML APIs. Flask: minimal and flexible. Django: batteries-included full-stack framework with ORM and admin.

Q. What is Docker and why use it?

Docker packages an application with its dependencies in a container, ensuring the same behaviour across machines. Images are built from a Dockerfile; containers are running instances; docker-compose orchestrates multi-container apps (API + vector DB + Redis).

Q. What is Kubernetes?

A container orchestration system that handles deployment, scaling, self-healing and service discovery across a cluster of machines.

Q. What is CI/CD?

Continuous Integration automatically builds and tests code on every commit; Continuous Delivery/Deployment automatically releases it. Tools: GitHub Actions, GitLab CI, Jenkins.

Q. How do you scale an LLM application?

Horizontal scaling of stateless API servers behind a load balancer, async calls, request queues, rate limiting, caching, batching, autoscaling, model routing (small model first), and a managed or self-hosted inference server for heavy load.

Q. What is caching and what types help in AI apps?

Storing results to avoid recomputation: embedding cache, retrieval cache, exact-match response cache and semantic cache (return a stored answer for similar questions). Use TTLs and invalidation when documents change.

Q. How do you monitor an LLM application?

Track latency, token usage and cost, error rates, retrieval hit rate, user feedback, hallucination/faithfulness scores on sampled traffic, and drift in query topics. Use tracing tools such as LangSmith, Langfuse, Phoenix or OpenTelemetry.

Q. What is MLOps?

Practices for taking ML systems to production reliably: data and model versioning (DVC, MLflow), reproducible pipelines, automated testing, deployment, monitoring and retraining. LLMOps extends this to prompts, evaluation sets and guardrails.

Q. What is rate limiting and why is it needed?

Restricting how many requests a client can make in a time window to protect services from abuse and control cost. Implement with token bucket or sliding window algorithms and return HTTP 429.

Q. What is authentication vs authorization?

Authentication verifies who you are (login, JWT, OAuth); authorization determines what you are allowed to do (roles, permissions). In RAG, authorization must be applied when retrieving documents.

Q. What is WebSocket / Server-Sent Events and why for chat?

SSE and WebSockets allow the server to push tokens to the client as they are generated, so users see streaming answers instead of waiting for the full response.

Q. What is a message queue?

Middleware (RabbitMQ, Kafka, SQS) that decouples producers and consumers, enabling asynchronous background jobs such as document ingestion and embedding.

Q. How do you store secrets such as API keys?

Environment variables, .env files excluded from Git, and secret managers (AWS Secrets Manager, Azure Key Vault, HashiCorp Vault). Never hardcode or commit keys; rotate them if leaked.

Q. Monolith vs microservices?

A monolith is a single deployable unit - simpler to start. Microservices split functionality into independently deployable services - better scaling and team autonomy but more operational complexity. Start simple.

Q. How would you design a URL shortener or chat system?

Clarify requirements, estimate scale, define APIs and data model, choose storage (key-value for URL mapping), generate unique IDs (base62 of a counter or hash), add caching and CDN, handle collisions, analytics, replication and failure handling. Always discuss trade-offs.

12. Git, Cloud, Linux and Networking Basics

Q. Git merge vs rebase?

Merge combines branches with a merge commit and preserves history. Rebase replays your commits on top of another branch, giving a linear history; avoid rebasing shared/public branches.

Q. Common Git commands?

git init, clone, status, add, commit, branch, checkout/switch, merge, rebase, pull, push, fetch, stash, log, diff, reset, revert, cherry-pick.

Q. git pull vs git fetch?

fetch downloads remote changes without merging; pull is fetch followed by merge (or rebase).

Q. git reset vs git revert?

reset moves the branch pointer back (rewrites history); revert creates a new commit that undoes a previous commit (safe for shared branches).

Q. What is a pull request?

A request to merge your branch into another, enabling code review, discussion and automated checks before merging.

Q. What is the difference between IaaS, PaaS and SaaS?

IaaS gives virtual infrastructure (EC2). PaaS gives a managed platform to run apps (Heroku, App Service). SaaS gives complete software (Gmail).

Q. Name key AWS services for an AI app.

EC2/ECS/EKS (compute), Lambda (serverless), S3 (storage), RDS/DynamoDB (databases), SageMaker (ML), Bedrock (managed foundation models), OpenSearch (search/vector), CloudWatch (monitoring), IAM (access).

Q. Important Linux commands?

ls, cd, pwd, cp, mv, rm, mkdir, cat, grep, find, chmod, chown, ps, top, kill, df, du, tail -f, curl, ssh, scp, tar, sudo, systemctl.

Q. What is the difference between TCP and UDP?

TCP is connection-oriented and reliable (ordering, retransmission) - used for web and APIs. UDP is connectionless and faster with no delivery guarantee - used for streaming and gaming.

Q. What happens when you type a URL in the browser?

DNS resolves the domain to an IP, a TCP connection (and TLS handshake for HTTPS) is established, the browser sends an HTTP request, the server responds with content, and the browser renders it.

Q. HTTP vs HTTPS?

HTTPS is HTTP over TLS, encrypting traffic and verifying the server identity with certificates.

Q. What is a CDN?

A Content Delivery Network caches content on servers close to users to reduce latency and load on the origin.

Q. What is a load balancer?

A component that distributes incoming traffic across multiple servers to improve availability and throughput, with strategies like round robin, least connections and IP hash.

13. Computer Science Core (OS, DBMS, OOP, Networks)

Q. What is an operating system?

Software managing hardware and providing services to programs: process management, memory management, file systems, device I/O and security.

Q. What is deadlock and its four conditions?

Processes wait forever for resources held by each other. Conditions: mutual exclusion, hold and wait, no preemption and circular wait. Break any one to prevent deadlock.

Q. What is virtual memory and paging?

Virtual memory gives each process the illusion of a large contiguous address space, mapping to physical memory in fixed-size pages and swapping to disk when needed. A page fault occurs when a needed page is not in RAM.

Q. What is a semaphore vs mutex?

A mutex is a lock owned by one thread for mutual exclusion. A semaphore is a counter allowing up to N concurrent accesses and is also used for signalling.

Q. Process scheduling algorithms?

FCFS, SJF, Priority, Round Robin, Multilevel Queue. Round Robin gives each process a fixed time slice, ensuring fairness.

Q. What are the four pillars of OOP?

Encapsulation, abstraction, inheritance and polymorphism.

Q. What are SOLID principles?

Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion - guidelines for maintainable object-oriented design.

Q. What are common design patterns?

Creational (Singleton, Factory, Builder), Structural (Adapter, Decorator, Facade), Behavioural (Observer, Strategy, Command). Example: the retriever interface in RAG frameworks is a Strategy pattern.

Q. What are the OSI layers?

Physical, Data Link, Network, Transport, Session, Presentation, Application (remember: Please Do Not Throw Sausage Pizza Away). TCP/IP simplifies this into four layers.

Q. What is the difference between a compiler and an interpreter?

A compiler translates the whole program to machine code before execution; an interpreter executes code line by line. Python uses both: it compiles to bytecode, then the VM interprets it.

14. Statistics and Math for Data Science

Q. Mean, median, mode - when to use which?

Mean for symmetric data, median for skewed data or outliers (income), mode for the most frequent category.

Q. What is standard deviation and variance?

Variance is the average squared deviation from the mean; standard deviation is its square root, measuring spread in the original units.

Q. What is the normal distribution and the 68-95-99.7 rule?

A symmetric bell-shaped distribution defined by mean and standard deviation; about 68, 95 and 99.7 percent of values lie within 1, 2 and 3 standard deviations.

Q. What is the Central Limit Theorem?

The distribution of sample means approaches a normal distribution as the sample size grows, regardless of the population's distribution (given finite variance).

Q. What is a p-value?

The probability of observing results at least as extreme as the data if the null hypothesis were true. A small p-value (commonly < 0.05) suggests evidence against the null; it is not the probability that the null is true.

Q. Type I vs Type II error?

Type I: rejecting a true null (false positive). Type II: failing to reject a false null (false negative).

Q. What is correlation vs causation?

Correlation measures association; causation means one variable produces change in another. Correlation does not imply causation; use controlled experiments or causal inference.

Q. What is Bayes' theorem?

P(A|B) = P(B|A) * P(A) / P(B). It updates the probability of a hypothesis given new evidence; the foundation of Naive Bayes.

Q. What is an A/B test?

A controlled experiment comparing two variants with randomly assigned users to determine if a change has a statistically significant effect on a metric.

Q. Dot product, cosine similarity and matrix multiplication - why do they matter?

Embeddings are vectors; similarity search uses the dot product or cosine similarity; neural networks are mostly matrix multiplications. Cosine = (A . B) / (|A| |B|).

Q. What is entropy?

A measure of uncertainty or impurity in a distribution: H = -sum p log p. Used in decision trees and in cross-entropy loss.

15. Projects, Resume and Practical Tips

Q. How should I explain my RAG chatbot project?

Use the framework: Problem - Solution - Architecture - Your role - Challenges - Results. Example: "Users could not quickly find answers in large PDFs. I built a RAG chatbot that ingests PDFs, chunks and embeds them, retrieves with hybrid search and answers with citations. I owned ingestion, retrieval and evaluation. Hallucinations were my main challenge; adding a relevance threshold and a strict prompt reduced unsupported answers significantly. Faithfulness improved from 0.71 to 0.88 on my test set."

Q. What questions will the interviewer ask on my project?

Expect: why this architecture, which embedding model and why, chunk size and overlap, vector DB choice, how you evaluated, how you handle hallucinations, latency/cost numbers, scaling plans, security, what failed, what you would improve and what you personally implemented. Prepare a short answer plus numbers for each.

Q. How should the resume look for an AI/ML fresher?

One page, clean format, sections in this order: summary, skills, projects (with impact numbers and tech stack), experience/internships, education, achievements, links (GitHub, LinkedIn). Use action verbs and measurable results; keep it ATS-friendly.

Q. How should I prepare for a technical interview in 2 weeks?

- Days 1-3: revise Python, SQL and core CS basics.
- Days 4-6: DSA practice daily (arrays, strings, hash maps, trees, DP basics).
- Days 7-9: ML/DL concepts and your project internals.
- Days 10-11: LLM and RAG concepts, system design basics.
- Days 12-13: mock interviews, HR answers, STAR stories.
- Day 14: light revision, company research and good sleep.

Q. How do I answer a question I do not know?

Stay calm, say what you do know, reason aloud from first principles, state your assumptions and be honest: "I have not used that directly, but I understand it is similar to X, and here is how I would approach it." Honesty plus a structured approach beats guessing.

Q. What should I do during a coding interview?

Clarify the problem and constraints, discuss examples and edge cases, outline the brute-force solution, then optimise, write clean code while thinking aloud, test with examples and state time/space complexity.

Q. What do interviewers look for in a fresher?

Strong fundamentals, clear communication, problem-solving approach, curiosity, projects you truly understand, honesty about gaps and the ability to learn quickly.

Q. How do I negotiate a job offer politely?

Express gratitude and enthusiasm, share market data or competing context, ask whether there is flexibility on compensation or benefits, keep the tone collaborative and be ready to explain your value. Get the final offer in writing.

Q. Dress code and etiquette for interviews?

Dress neatly for the company culture, join early, test camera/mic and internet for online interviews, keep a quiet well-lit background, maintain eye contact (look at the camera), listen fully before answering and send a thank-you email afterwards.

Q. What are good questions to ask the interviewer about AI work?

"How do you evaluate model or RAG quality in production?", "What is the biggest technical challenge your team faces?", "How do you handle data privacy for LLM features?", "How are research ideas turned into products here?"

16. General and Situational Questions (Non-Coding)

Q. Why did you choose this career field?

Share a genuine story: what sparked your interest, how you explored it (courses, projects, internships) and why it still excites you. Link it to the role you are applying for.

Q. What do you know about our company?

Mention what the company does, its products or customers, recent news or launches, and its values or culture. Then connect one point to your own interests: "I like that you focus on X, which matches my work on Y."

Q. What is your biggest fear or what stresses you the most?

Be honest but constructive: "I get stressed when expectations are unclear, so I ask questions early and confirm priorities." Show you have a method to manage it.

Q. How would you describe yourself in three words?

Pick three relevant words and give one-line proof for each, e.g. "Curious (I teach myself new tools), reliable (I meet deadlines), collaborative (I help teammates debug)."

Q. What would your friends or colleagues say about you?

Mention positive, believable traits and a small example. Example: "They would say I am the person who stays calm and helps break big problems into steps."

Q. Tell me about a time you showed leadership.

Leadership need not mean a title. Describe a time you organised a team, took initiative, or guided others. Use STAR and highlight how you motivated people and what outcome the team achieved.

Q. Tell me about a time you worked in a team.

Describe the goal, your specific role, how the team communicated, how you handled differences and the result. Give credit to teammates while being clear about your own contribution.

Q. Tell me about a time you had to learn something quickly.

Pick a recent example (a new framework, tool or concept). Explain your learning plan - docs, small experiments, asking experts - the time taken and what you built or delivered with it.

Q. Tell me about a time you made a mistake at work or college.

Own the mistake without blaming others, describe how you fixed it, what you learnt and what process you changed so it does not repeat.

Q. How do you handle criticism or feedback?

Listen without defending, ask clarifying questions, thank the person, apply the useful parts and follow up later to show improvement. Share one example where feedback made your work better.

Q. What would you do if you are given a task you have never done before?

Break the task down, research quickly, look at similar examples, ask a teammate for direction on the unknowns, build a small prototype, get early feedback and document what you learn.

Q. What would you do if you cannot meet a deadline?

Inform the stakeholders early, explain the reason, share a realistic new date or a reduced scope, ask for help if useful and focus on delivering the highest-priority part first.

Q. What if you disagree with a team decision?

Present your concern with data respectfully, listen to the reasoning, and if the team still decides otherwise, commit to the decision and support its success.

Q. What would you do if you find a mistake in your senior's work?

Verify first, then raise it privately and politely as a question or observation ("I noticed X, could you confirm?"), focusing on the work and not the person.

Q. What would you do if you are overloaded with work?

List all tasks, rank by urgency and impact, discuss priorities with your manager, delegate or defer what is possible and communicate clearly instead of silently missing deadlines.

Q. How do you deal with a difficult colleague or customer?

Stay calm and respectful, listen to understand their concern, find common ground, propose solutions and escalate only if the issue remains unresolved.

Q. What do you do when you have nothing to do at work?

Ask if teammates need help, improve documentation or tests, learn something relevant to the team's work and suggest improvements.

Q. Would you prefer a bigger company or a startup?

Show balance: big companies offer structure and mentoring; startups offer ownership and speed. Say what matches you now and that you are eager to learn in either.

Q. How do you define success?

Delivering real value, continuous growth and a positive impact on the team and users, rather than only titles or salary.

Q. What are your hobbies and interests?

Share genuine hobbies and what they teach you (sports - teamwork, reading - curiosity, music - discipline). Keep it short and positive.

Q. Tell me something that is not on your resume.

Share a short personal story about a hobby, volunteering, a challenge you overcame or a skill you learnt on your own, which shows character and values.

Q. Is there anything else you would like to add?

Use this moment to briefly restate why you are a strong fit and your enthusiasm: "I am excited about the role and confident that my skills and learning attitude will help the team from day one."

Q. Do you have any other job offers or interviews?

Be honest and professional. You may say you are exploring options but that this role is a strong priority because of its fit and growth opportunities.

Q. Why is there a gap in your resume/education?

Be honest and brief: state the reason (health, family, preparation, upskilling) and what you did productively during the gap, then pivot to your readiness now.

Q. Why did you get low marks in some subjects or semesters?

Take responsibility, explain the circumstances briefly without excuses, and show improvement through later results, projects or certifications.

Q. How would you sell me this pen? (sales-style question)

Ask about needs first (who uses it, how often), then match features to benefits: smooth writing, durability, comfort. Close by confirming the need is met. Interviewers check structure and communication, not product knowledge.

Q. Can you work long hours or weekends if required?

Be honest: you are flexible for critical deadlines while valuing sustainable productivity. Ask what the usual expectation is.

Q. How do you manage your time and avoid procrastination?

Plan the day, use priorities, work in focused blocks (Pomodoro), remove distractions, set small deadlines and review progress each evening.

Q. What is your learning style?

Learn by doing: read the concept, build something small, explain it to someone else and revisit it. Give an example of a topic learnt that way.

17. Aptitude, Logical and Puzzle Questions

Q. How do you approach a puzzle you have never seen before?

Restate the problem, note constraints, try small cases, look for patterns, break it into parts and think aloud. Interviewers care about reasoning more than the final answer.

Q. Estimation question: how many tennis balls fit in a school bus?

Use a Fermi estimate: bus volume (about 12 m x 2.5 m x 2 m = 60 m^3), ball volume (about 0.00023 m^3 each), packing efficiency (about 64 percent). Balls = 60 x 0.64 / 0.00023, roughly 165,000. State assumptions clearly and sanity-check the result.

Q. Estimation question: how many smartphones are sold in India per year?

Estimate population (about 1.4 billion), share of smartphone users, average replacement cycle (about 3 years): roughly 700 million users / 3 = around 230 million per year. The exact number is not important; the structured approach is.

Q. There are 3 bulbs inside a room and 3 switches outside. How do you identify which switch controls which bulb with only one entry?

Turn switch 1 on for several minutes, then turn it off. Turn switch 2 on and enter. The lit bulb is switch 2, the warm unlit bulb is switch 1 and the cold unlit bulb is switch 3.

Q. You have 8 balls, one is heavier. Find it with the fewest weighings.

Two weighings. Weigh 3 vs 3: if unequal, the heavy one is in that group; if equal, it is among the remaining 2. Then weigh 1 vs 1 from the suspect group (for a group of 3) to find it. (With 9 balls, 2 weighings as well.)

Q. A train 100 m long crosses a pole in 10 seconds. What is its speed?

Speed = distance / time = 100 / 10 = 10 m/s = 36 km/h.

Q. If 5 machines make 5 items in 5 minutes, how long do 100 machines take to make 100 items?

5 minutes. Each machine makes 1 item in 5 minutes, so 100 machines make 100 items in the same 5 minutes.

Q. A bat and a ball cost 110 rupees in total. The bat costs 100 more than the ball. How much is the ball?

5 rupees. Ball = x, bat = x + 100, total = 2x + 100 = 110, so x = 5. (The common wrong answer is 10.)

Q. What is the probability of getting at least one head in two coin tosses?

3/4. Total outcomes: HH, HT, TH, TT; only TT has no head. 1 - 1/4 = 3/4.

Q. What is the probability of getting a sum of 7 with two dice?

6/36 = 1/6. Favourable pairs: (1,6), (2,5), (3,4), (4,3), (5,2), (6,1).

Q. Find the next number: 2, 6, 12, 20, 30, ?

42. The differences are 4, 6, 8, 10, then 12, so 30 + 12 = 42. (The terms are n x (n + 1).)

Q. A clock shows 3:15. What is the angle between the hands?

7.5 degrees. The hour hand is at 3 x 30 + 15 x 0.5 = 97.5 degrees, the minute hand at 15 x 6 = 90 degrees; the difference is 7.5.

Q. Percentage: a price rises by 20 percent then falls by 20 percent. What is the net change?

A 4 percent decrease. 1.2 x 0.8 = 0.96.

Q. Simple interest on 10,000 rupees at 8 percent for 3 years?

SI = P x R x T / 100 = 10,000 x 8 x 3 / 100 = 2,400 rupees.

Q. A can do a job in 10 days and B in 15 days. Working together, how long?

6 days. Combined rate = 1/10 + 1/15 = 1/6 per day.

Q. Two doors puzzle: one leads to freedom, one to danger. One guard always lies, one always tells the truth. What single question do you ask?

Ask either guard: "If I asked the other guard which door leads to freedom, what would he say?" Then choose the opposite of the answer, because it is always the wrong door.

Q. How would you explain a complex topic (like RAG) to a non-technical person?

Use an analogy: "It is like an open-book exam. Instead of answering from memory, the assistant first looks up the relevant pages in your documents and then writes the answer from them." Keep jargon out and check for understanding.

18. Current Technology, AI Awareness and Ethics

Q. What do you think about the future of AI?

Balanced view: AI will automate routine tasks, boost productivity and create new roles, but needs responsible development, human oversight and continuous reskilling. Say you plan to keep learning and use AI as a tool.

Q. Will AI replace jobs?

AI will change many jobs and automate certain tasks, but people who learn to work with AI will be more valuable. New roles in AI engineering, evaluation, safety and domain-specific applications are emerging.

Q. What are the ethical concerns in AI?

Bias and fairness, privacy and data protection, transparency/explainability, misinformation and deepfakes, copyright, security misuse, environmental cost and accountability. Mitigate with diverse data, audits, human-in-the-loop, clear policies and regulation.

Q. What is responsible AI?

Designing and deploying AI that is fair, transparent, accountable, safe and privacy-preserving, with documented limits, monitoring and human oversight.

Q. Difference between AI, ML, Deep Learning and Generative AI?

AI is the broad goal of intelligent behaviour; ML learns from data; Deep Learning uses multi-layer neural networks; Generative AI creates new content (text, images, code, audio) using models such as LLMs and diffusion models.

Q. What is ChatGPT and how does it work in simple words?

It is a chatbot powered by a large language model that predicts the next word repeatedly, trained on huge amounts of text and then tuned with human feedback to follow instructions helpfully and safely.

Q. What is data privacy and which laws exist?

Protecting personal data from misuse. Examples: GDPR (EU), DPDP Act 2023 (India), CCPA (California). Practices include consent, minimisation, encryption, access control and deletion on request.

Q. What is cybersecurity awareness for an employee?

Use strong unique passwords and MFA, avoid phishing links, keep software updated, do not share confidential data with unapproved tools (including public AI chatbots) and report incidents quickly.

Q. Should employees paste company data into public AI tools?

No, not without approval. Confidential data may be stored or reused by third parties. Use approved enterprise tools or private deployments and follow the company's data policy.

Q. What are the latest trends in tech you follow?

Examples: LLM agents, RAG and evaluation, multimodal models, small/local models, vector search, AI governance, cloud-native platforms and edge AI. Mention a trend you have actually tried and what you learnt.

Q. Open-source vs proprietary models - which is better?

Proprietary APIs (GPT, Claude, Gemini) offer top quality and easy use; open models (Llama, Mistral, Qwen) offer control, privacy, customisation and lower long-term cost at scale. Choose by data sensitivity, budget, quality needs and team skills.

Q. What is cloud computing and its benefits?

Delivering computing resources over the internet on demand: scalability, pay-as-you-go cost, reliability, global reach and faster development.

Q. What is Digital India / Startup India / AI Mission (India context)?

Government initiatives for digital infrastructure and services, startup support and AI capacity building (compute, datasets, skilling). Know one or two facts and their impact on your field, and verify current details before the interview.

Q. What is blockchain, IoT and 5G in one line each?

Blockchain: tamper-resistant distributed ledger. IoT: internet-connected physical devices that collect and exchange data. 5G: fifth-generation mobile network with higher speed, lower latency and more device capacity.

Q. How do you verify information you read online or from an AI?

Check the original source, cross-verify with reliable sources, look at the date and author, and be especially careful with numbers, quotes and medical/legal claims. Treat AI output as a draft that needs verification.

19. Communication, English and Interview Etiquette

Q. How can I improve my communication in interviews?

Practise speaking answers aloud, record yourself, use simple clear sentences, structure answers (point - reason - example), avoid filler words, slow down, and listen fully before answering.

Q. How to introduce myself in a formal way?

"Good morning. Thank you for this opportunity. I am [Name], a [degree/role] from [college/company]. My core skills are [skills], and recently I built [project]. I am keen to contribute as [role] at [company]."

Q. What if I do not understand the question?

Politely ask: "Could you please repeat or rephrase the question?" or paraphrase it to confirm: "Just to confirm, you are asking about...?" This shows clarity of thought.

Q. What if my English is not perfect?

Clear and confident communication matters more than perfect grammar. Use simple words, speak slowly, and it is fine to pause and think. Practise common phrases and keep improving by speaking daily.

Q. Useful phrases to use during an interview?

"That is a great question." "Let me think for a moment." "From my experience..." "One example is..." "To summarise..." "I would approach it in three steps." "I am not sure, but my understanding is..."

Q. What body language should I maintain?

Sit upright, smile naturally, maintain eye contact (look at the camera online), use calm hand gestures, avoid fidgeting and nod to show you are listening.

Q. How do I handle nervousness?

Prepare well, take slow deep breaths, pause before answering, treat the interview as a conversation, drink water and remember that interviewers want you to succeed.

Q. How should I follow up after the interview?

Send a short thank-you email within 24 hours: thank them, mention one point from the discussion, restate your interest and fit, and keep it polite and brief.

Q. Online interview checklist?

Stable internet, charged laptop, camera at eye level, good lighting, quiet room, headphones, plain background, documents ready, notifications off and the meeting link tested 10 minutes early.

Q. What documents should I carry for an in-person interview?

Several resume copies, government ID, educational certificates and marksheets, experience letters and offer letters if any, a notepad and pen, and a portfolio or project list.

Q. How do I answer "Are you comfortable with the work culture here?"

Say you have researched the culture, you are adaptable, you value teamwork and learning, and you would be glad to hear more about how the team works day to day.

Q. How do I answer "Tell me about a time you went above and beyond"?

Use STAR: describe a situation where you did more than expected (extra testing, helping a teammate, improving a process), the effort you made and the measurable benefit.

Appendix A: Quick Revision Cheat Sheet

Topic	Remember this
RAG pipeline	Load > Chunk > Embed > Index > Retrieve > Rerank > Prompt > Generate > Evaluate
Chunk size	200-800 tokens, 10-20% overlap; tune on your own data
Retrieval	Hybrid (BM25 + vector) + reranker; top-k 3-8 after reranking
Hallucination fix	Grounded prompt, threshold, citations, low temperature, evaluation
RAG metrics	Faithfulness, answer relevancy, context precision, context recall, Recall@k, MRR
Bias vs variance	Underfit = high bias; overfit = high variance
Precision / Recall	TP/(TP+FP) / TP/(TP+FN); F1 = harmonic mean
Big-O	O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n)
SQL order	FROM > WHERE > GROUP BY > HAVING > SELECT > ORDER BY > LIMIT
ACID	Atomicity, Consistency, Isolation, Durability
STAR	Situation, Task, Action, Result
Transformer	Self-attention = softmax(QK^T / sqrt(dk)) V
LoRA	Train small low-rank adapters, freeze the base model
HTTP codes	200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests, 500 Server Error

Appendix B: Glossary

Embedding: Numeric vector representing the meaning of text.

Chunk: A small piece of a document stored and retrieved as one unit.

Vector database: Database specialised in storing embeddings and similarity search.

Retriever: Component that returns relevant documents for a query.

Reranker: Model that re-scores retrieved chunks for better relevance.

Hallucination: Confident but unsupported or false model output.

Token: Sub-word unit processed by an LLM; used for billing and limits.

Context window: Maximum tokens a model can read and write in one request.

Prompt: Instruction and context given to the model.

Fine-tuning: Further training of a model on task-specific data.

LoRA: Parameter-efficient fine-tuning using low-rank adapters.

Quantization: Lowering numeric precision to reduce memory and speed up inference.

ANN: Approximate nearest neighbour search for fast vector lookup.

HNSW: Graph-based ANN index with high recall.

BM25: Keyword ranking function used in search engines.

RRF: Reciprocal Rank Fusion, a method to merge ranked lists.

Agent: LLM system that chooses tools and actions iteratively.

Guardrails: Rules and filters that keep model behaviour safe and on-topic.

RLHF: Alignment training using human preference feedback.

MCP: Open protocol for connecting AI apps to tools and data.